ADVANCED PHASE · DAY 24 OF 30

Routes, path & query parameters, Pydantic validation, async endpoints, and the auto-generated Swagger UI

2 hrs	3	24	Advanced
SESSION LENGTH	CONCEPT FILES	PRACTICE PROBLEMS	PHASE

30-Day Python Programming Course — Instructor Session Guide

Session Overview

Goal: Give students hands-on fluency building a real backend API with FastAPI — defining routes, validating input with Pydantic, and understanding why/when to reach for `async def`.

Outcome: By the end of this session, students will have built and tested a small "TiffinExpress" food-delivery backend covering path parameters, query parameters, request bodies, Pydantic validation, async endpoints, and will know how to read the auto-generated Swagger docs at `/docs`.

Timeline

Time	Segment	Deliverable
0:00 - 0:10	Recap Day 23 + intro FastAPI	Discussion
0:10 - 0:35	Routes & path parameters	day24_01_routes_path_params.py
0:35 - 1:05	Query params, request bodies & Pydantic	day24_02_query_params_pydantic.py
1:05 - 1:15	Break	-
1:15 - 1:45	Async endpoints & Swagger UI	day24_03_async_swagger_docs.py
1:45 - 2:00	Practice problems + wrap-up	day24_practise_problems_*.py

> **TIP**
>
> FastAPI apps in this course are demonstrated with `TestClient` instead of a live `uvicorn` server, so every file runs cleanly top-to-bottom with a single `python filename.py` command — no second terminal or browser required during the session.

Prerequisites & Preparation

Requirements

Requirement	Why it's needed
Python 3.12 + uv (Day 1)	Environment already set up
Comfort with type hints (Day 18)	FastAPI relies entirely on type hints
requests / REST concepts (Day 23)	Today we build what Day 23 consumed
pip install fastapi uvicorn pydantic httpx2	Core dependencies for this session

Instructor Pre-Session Checklist

```
[ ] Confirm fastapi, uvicorn, pydantic, and httpx2 are installed in the student environment
[ ] Pre-run all three concept files once to warm up caches / catch environment issues
[ ] Have the TiffinExpress domain (restaurants, orders, menu items) recapped from Day 23 ready to reference
[ ] Optionally prepare ONE live uvicorn demo of /docs in a browser for visual impact
[ ] Review the route-ordering gotcha (Section 5, File 1) - a common source of confusion
```

Step-by-Step Instruction Guide

STEP 1 - Installing FastAPI & First Route

What to Say

"Today we flip the script from Day 23. Instead of calling someone else's API, you're going to BUILD one. FastAPI lets us describe an API almost entirely using Python type hints — and it gives us free validation and free documentation just for doing that."

```
# Install dependencies (one-time)
pip install fastapi uvicorn pydantic httpx2 --break-system-packages

from fastapi import FastAPI
from fastapi.testclient import TestClient

app = FastAPI(title="TiffinExpress API")

@app.get("/")
def read_root():
    return {"service": "TiffinExpress API", "status": "running"}

client = TestClient(app)
print(client.get("/").json())
# {'service': 'TiffinExpress API', 'status': 'running'}
```

Error	Cause	Fix
ModuleNotFoundError: fastapi	Dependencies not installed	Re-run the pip install command with --break-system-packages
StarletteDeprecationWarning re: httpx	httpx2 not installed alongside fastapi	pip install httpx2
404 on every route	Forgot the @app.get(...) decorator	Check every route function has a decorator directly above it

DONE WHEN

Students see a JSON response printed to the console with zero manual `json.dumps()` calls, and understand that FastAPI converts return values to JSON automatically.

STEP 2 - Path Parameters & Route Ordering

What to Say

“Notice how the URL segment in curly braces becomes a function argument. Add a type hint, and FastAPI validates it FOR you — try requesting a restaurant with a non-numeric ID and watch what happens.”

```
@app.get("/restaurants/{restaurant_id}")
def get_restaurant(restaurant_id: int):
    ...

client.get("/restaurants/not-a-number")
# status_code: 422
# FastAPI auto-generates a validation error - no manual check needed
```

WATCH OUT

Define static/specific paths (like `/restaurants/featured`) **before** parameterized ones (like `/restaurants/{restaurant_id}`) or the specific route will never be reached.

DONE WHEN

Students correctly predict a 422 status code before running the code, and can explain WHY route order matters using their own words.

STEP 3 - Query Params, Request Bodies & Pydantic

What to Say

“Any parameter that ISN'T named in the path becomes a query parameter automatically. And when we want to accept a full JSON object — like an order with multiple items — we describe its shape with a Pydantic model instead of a plain dict.”

```
class OrderItem(BaseModel):
    item_name: str
    quantity: int = Field(gt=0)
    price_inr: float = Field(gt=0)

class OrderCreate(BaseModel):
    customer_name: str
    restaurant_id: int
    items: list[OrderItem]

@app.post("/orders")
def place_order(order: OrderCreate):
    total = sum(i.quantity * i.price_inr for i in order.items)
    return {"total_inr": total}

# Sending quantity=-1 is rejected automatically:
# status_code: 422 (never reaches our function body)
```

Error	Cause	Fix
422 on a POST that looks correct	Field name mismatch between JSON and model	Compare JSON keys exactly against the Pydantic model's field names
TypeError: unexpected keyword	Model field missing a default but not sent	Add a default value or send the field explicitly
Query param ignored	Same name accidentally also appears in path	Path names always win - rename one of them

DONE WHEN

Students can place a valid order via TestClient AND trigger a validation error on purpose (negative quantity), explaining the 422 response in their own words.

STEP 4 - Async Endpoints & Swagger UI

What to Say

“Remember asyncio from Day 16? FastAPI is built on it. If a route waits on something slow — a database call, another API — mark it async def and await the slow part, so FastAPI can serve OTHER requests during

the wait. And the best part: you get a full interactive API explorer at /docs for free, generated from the exact code you already wrote."

```
async def fetch_delivery_partner_eta(restaurant_id: int) -> int:
    await asyncio.sleep(0.05)  # simulates a network call
    return eta_lookup.get(restaurant_id, 35)

@app.get("/restaurants/{restaurant_id}/eta")
async def get_delivery_eta(restaurant_id: int):
    eta = await fetch_delivery_partner_eta(restaurant_id)
    return {"restaurant_id": restaurant_id, "eta_minutes": eta}

# To view Swagger UI live (optional demo):
uvicorn main:app --reload
# then open http://127.0.0.1:8000/docs
```

TIP

If projecting a live demo, open both `/docs` and `/redoc` side by side so students see the same schema rendered two different ways.

DONE WHEN

Students can articulate WHEN to use async def (I/O-bound waits) versus a plain def route, and locate their own routes inside the /openapi.json schema.

Session Wrap-Up

Student Checklist

```
[ ] Built and ran all three concept files without errors
[ ] Can explain why route ORDER matters for static vs parameterized paths
[ ] Defined at least one Pydantic model with a Field() constraint
[ ] Wrote one async def route and awaited a helper function inside it
[ ] Attempted the Day 24 practice problem set (24 problems across 3 files)
```

Key Concepts Recap

Concept	One-Line Summary
Path parameters	Variable URL segments, auto-typed & validated via type hints
Query parameters	Extra function args not named in the path; optional via defaults
Pydantic models	Class-based schemas for request bodies with built-in validation
Field() constraints	Per-field rules like gt, ge, le enforced before your code runs
async def routes	Let FastAPI serve other requests while awaiting slow I/O
/docs & /redoc	Auto-generated interactive documentation, zero manual writing

Next Day Preview

Day 25 — Databases & Packaging: We replace today's in-memory lists/dicts with a real SQLite database via SQLAlchemy ORM, structure the project with `pyproject.toml`, and containerize it with Docker. This directly sets up the Day 26-30 capstone: the full-stack Task Manager API.

Troubleshooting notes for next time: If students hit dependency conflicts between fastapi/pydantic versions, pin versions in a requirements file before Day 25's SQLAlchemy setup to avoid compounding issues.

Quick Reference Card

Daily Workflow Commands

```
# Install dependencies
pip install fastapi uvicorn pydantic httpx2 --break-system-packages

# Run any Day 24 file (uses TestClient, no live server needed)
python day24_01_routes_path_params.py

# Optional: run a REAL live server to view Swagger UI
uvicorn main:app --reload
# then visit http://127.0.0.1:8000/docs
```

VS Code Shortcuts

Shortcut	Action
Ctrl+` (Cmd+` on Mac)	Toggle integrated terminal
Shift+Enter	Run selected line(s) in terminal
F5	Start debugging the current file
Ctrl+Space	Trigger IntelliSense (great for Pydantic fields)

Tool Summary

Tool	Purpose
fastapi	Web framework - routes, validation, auto docs
pydantic	Data validation via typed models (BaseModel, Field)
uvicorn	ASGI server used to run a FastAPI app live
fastapi.testclient.TestClient	Call routes directly in scripts/tests, no live server
httpx2	Required alongside fastapi for warning-free TestClient use